



VIT[®]
AP



WEB TECHNOLOGIES

CSE-4004

MODULE 6: Node JS & Express JS



Express



By: Dr.MD.Sirajuddin

SCOPE

WHAT IS NODE JS?

- **NODE JS**, is an open source JavaScript runtime environment.
- It was originally created by **Ryan Dahl** in 2009
- It allows users to create servers, web apps, command line tools and scripts.
- It is cross platform runtime environment that allows us to execute JavaScript code outside the browser.
- It enables "**Full-Stack JavaScript**" where developers can use one single language across the entire application.

CORE FEATURES OF NODE JS

Features	Description
Asynchronous & Non Blocking	Node.js does not wait for tasks to finish. <pre>console.log("Start"); setTimeout(() => { console.log("Task done"); }, 2000); console.log("End");</pre>
Event Driven	- It works on Events. - User request → event → server responds
Single Threaded	uses only one main thread to execute code.
Very Fast Execution	- Built on Google Chrome's V8 engine. - Convert JavaScript code into machine code quickly
Cross Platform	Run on any platform
Rich Ecosystem	Huge Collection of Libraries
Suitable of real-time applications	Chat applications, live notifications, online games etc

SCOPE



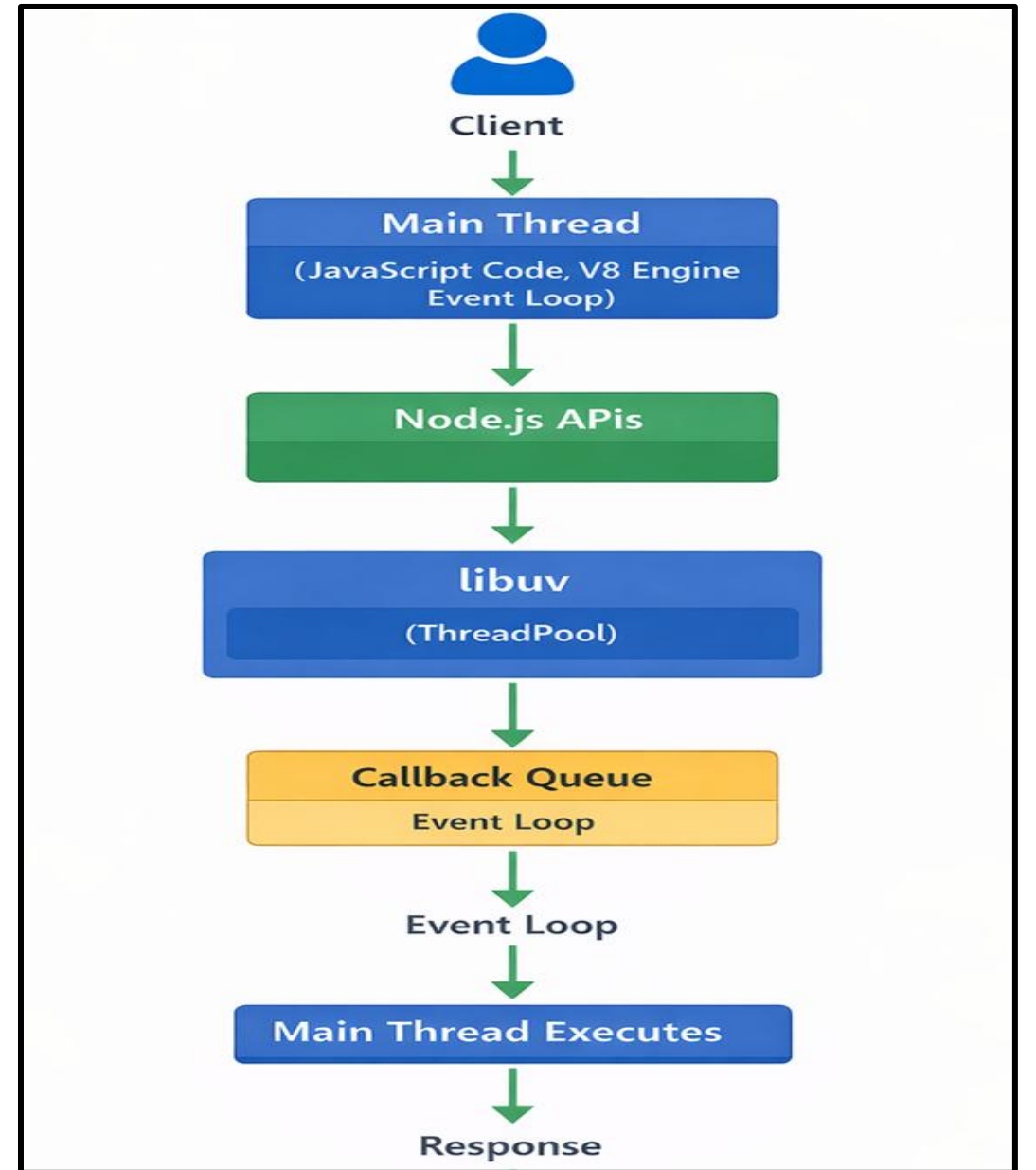
AP

CORE FEATURES OF NODE JS

Features	Description
JSON Support	• Works easily with JSON data
Faster Data Handling	• No buffering • Data is processed in chunks (streams)

Node JS Architecture

1. User Request comes from browser, API or client.
2. JS code is executed in main thread
3. If Async function is called then Node.js API is called, it sends task to Libuv Library
5. Libuv Library for background Processing
6. After task is completed it sends Callback to Callback Queue
7. Event Loop: Checks queue and takes callback & send it to the main thread
8. Callback is executed & Final output is displayed to User.



NODE JS INSTALLATION

1. **Install Node JS** → <https://nodejs.org/en/download>
 - **LTS Version + npm (node package manager)**
2. **Open Command prompt**
 - **C:\Users\Siraj> node -v**
v24.14.0
 - **C:\Users\Siraj> npm -v**
11.9.0
3. **Create project folder and then create file app.js**
4. **Open CMD prompt and go to the project folder**
5. **Run node app.js**

Creating Simple Server

1. Load built-in Node.js HTTP module.

```
const http = require("http"); //creates web server
```

2. Create Server

- `const server = http.createServer((req, res) => { });`
- `createServer()` creates a server object
- It takes a callback function
- This function runs every time a client sends a request

Parameters: (req, res)

- `req` → Request object : Contains client data
- `res` → Response object: used to send response to client.

3. Send Response: `res.write("Hello User");`

```
res.end("Hello from Node.js Server");
```

Note: If you don't call `res.end()`, browser will keep waiting

4. Start Server: `server.listen(3000, () => {callback});` callback run once the server is started

Creating Simple Server: Execution Process

- 1.Run file → node app.js
- 2.Server starts on port 3000
- 3.Browser sends request (<http://localhost:3000>)
- 4.Node.js receives request
- 5.Callback (req, res) executes
- 6.res.end() sends response
- 7.Browser displays output

Creating Simple Server

```
1. const http = require("http");  
2. const server = http.createServer((req, res) => {  
3.   res.write("Hello\n");  
4.   res.write("Welcome\n");  
5.   res.end("Final message from Node.js Server"); } );  
6. server.listen(3000, ( ) => {  
7.   console.log("Server running at http://localhost:3000");  
8. });
```

Execution:

1. Execute this code using `node server.js`
2. Open browser and type the url : <http://localhost:3000>

Command Line Arguments

1. Node.js provides a built-in array: `process.argv` that stores all arguments.
2. `console.log(process.argv)`
3. `E:\Node Project>node server.js`
4. Server running at `http://localhost:3000`
5. [
6. '`C:\\Program Files\\nodejs\\node.exe`',
7. '`E:\\Node Project\\server.js`'
8.]
9. It returns path of node.js and path of your JS file.

Command Line Arguments

```
const num1 = parseInt(process.argv[2]);  
const num2 = parseInt(process.argv[3]);  
console.log("Sum:", num1 + num2);
```

Import and Export

- `require()` is a Node.js function used to import modules (built-in or user defined)
- `const http = require("http");`
- `const fs = require("fs");`

Export

- `module.exports = value;` //export data from file
- *To use import keyword, (Modern JS → import)*
- Use `.mjs` file
- `import fs from "fs";`
- `import` can be used in Node.js, but it requires enabling ES Modules using `.mjs` extension

Example:

- Create add.js

```
function add(a, b)
{
  return a + b;
}
```

```
module.exports = add;
```

- Create app.js (main file)

```
const add = require("./add");
```

```
const result = add(10, 5);
```

```
console.log("Sum:", result);
```

- Execution: node app.js

Export

1. `module.exports = function add(a, b) { return a + b; };`
2. `module.exports = {
 name: "Bob",
 age: 25
}; // exporting object`
3. `const pi = 3.14;
 module.exports = pi;`
4. `function add(a, b) { return a + b; }
function sub(a, b) { return a - b; }
module.exports = { add, sub };`
5. `class Person {
 constructor(name) {
 this.name = name; }
 }
 module.exports = Person;`

Reading and Writing File (Asynchronous & Non Blocking)

1. File system module is required to read a file

```
const fs = require("fs");
```

2. **fs.readFile(filename, encoding, callback)**

encoding = "utf-8" Or ascii (it is optional parameter) → without encoding returns buffered object (raw data)

3. **fs.writeFile(filename, data, callback)**

```
const fs = require('fs');
```

```
fs.writeFile('data.txt', 'Hello', (err) => {
```

```
if (err) throw err;
```

```
console.log("Success");
```

```
});
```

Reading and Writing File (Asynchronous)

```
const fs = require('fs');  
fs.readFile('data.txt', 'utf-8', (err, data) => {  
  if (err) throw err;  
  console.log(data);  
});
```

- **fs.appendFile(filename, data, callback)**

```
const fs = require('fs');  
const content = "\nThis is appended text";  
fs.appendFile('data.txt', content, (err) => {  
  if (err) throw err;  
  console.log("Data appended successfully");  
});
```

Reading and Writing File (Synchronous)

```
const fs = require("fs");
```

```
fs.writeFileSync("data.txt", "Hello Node");
```

```
const data = fs.readFileSync("data.txt", "utf-8");
```

```
console.log(data);
```

```
fs.appendFileSync(filename, data)
```

Reading and Writing File Using Promise

- `const fs = require("fs").promises; //promise based version`
- Promise object represents result of asynchronous operation.
- `const promise = new Promise((resolve, reject) => { // asynchronous work });`
- `(resolve, reject) => { }`
- `resolve()`: Called when task is successful
- `reject()`: Called when task fails
- **States of Promise Object:**
 1. Pending
 2. Resolved
 3. Rejected

Reading and Writing File Using Promise

```
const fs = require("fs").promises;
```

```
fs.readFile("data.txt", "utf-8")  
  .then((data) => {  
    console.log(data);  
  })  
  .catch((err) => {  
    console.log("Error");  
  });
```

Connecting UI to Node JS

Index.html

```
<!DOCTYPE html>  
<html>  
<body>  
<h2>Enter Name</h2>  
  <form action="http://localhost:3000" method="GET">  
    <input type="text" name="username" />  
    <button type="submit">Send</button>  
  </form>  
</body>  
</html>
```

Connecting UI to Node JS

Serve1.js

```
const http = require("http");// used to create server
const url = require("url"); //used to parse url
const server = http.createServer((req, res) => {
  const q = url.parse(req.url, true).query; //query to object
  const name = q.username;
  res.end("<h1>Hello "
    + name + "</h1>");
});
server.listen(3000, () => {
  console.log("Server running at http://localhost:3000");
});
```

```
req.url="/?username=Bob&passwd=1234"
query = {
  username: "Bob",
  password: "1234"
}
```

```
const http = require("http"); // handling post
const server = http.createServer((req, res) =>
{
  if (req.method === "POST" && req.url === "/submit")
  {
    let body = "";
    req.on("data", chunk => { body += chunk; }); //event listener
    req.on("end", () => {
      console.log("Raw data:", body);
    });
    const params = new URLSearchParams(body); //converts query string
                                              to key-value format
    const username = params.get("username");
    res.end("Hello " + username);
  }); }
```

SCOPE

```
else {  
    res.end("Send POST request from form");  
}  
});  
server.listen(3000, () => {  
    console.log("Server running on 3000");  
});
```

- **Login.html**

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h2>Login Form</h2>
```

```
<form method="POST" action="http://localhost:3000/submit">
```

```
<input type="text" name="username" placeholder="Username">
```

```
<br><br>
```

```
<input type="password" name="password" placeholder="Password">
```

```
<br><br>
```

```
<button type="submit">Login</button>
```

```
</form> </body> </html>
```

- **server.js**

```
const http = require("http");
const server = http.createServer((req, res) => {
  if (req.method === "POST" && req.url === "/submit")
  {
    let body = "";
    req.on("data", chunk => {
      body += chunk;
    });
    req.on("end", () => {
      const params = new URLSearchParams(body);
      const username = params.get("username");
      const password = params.get("password");
```

```
res.writeHead(200, { "Content-Type": "text/html" });
res.end(` <h1>Username: ${username}</h1>
        <h2>Password: ${password}</h2>` ); });

}
else
{
    res.end("Send POST request from form");
}

});

server.listen(3000);
```

EVENT HANDLING

1. Node.js is Event-Driven
2. It waits for events and reacts

STEPS:

1. `const EventEmitter = require("events");` → create events, listen to events & trigger event
2. **Create object** → `const emitter = new EventEmitter();`
3. Listen to the event → `emitter.on("greet", () => { console.log("Hello");});`
"When 'greet' happens, run this function"
4. Trigger an event → `emitter.emit("greet");`

```
1. const EventEmitter = require("events");  
2. const emitter = new EventEmitter();  
3. emitter.on("greet", () => {console.log("Hello User");});  
4. emitter.emit("greet");
```

```
const EventEmitter = require("events");  
const emitter = new EventEmitter();  
function display()  
{  
  console.log("Hello User");  
}  
emitter.on("greet", display);  
emitter.emit("greet");
```

Methods	Meaning
<code>on()</code>	Listen for an event (runs every time event happens)
<code>once()</code>	Listen only once (runs only first time)
<code>emit()</code>	Trigger / fire an event
<code>addListener()</code>	Same as <code>on()</code> (alternative name)
<code>removeListener()</code>	Remove a specific listener
<code>removeAllListeners()</code>	Remove all listeners of an event

Stream Events	Meaning	Example
<code>data</code>	Data is coming in chunks	<code>req.on("data", chunk)</code>
<code>end</code>	All data received	<code>req.on("end", function)</code>
<code>error</code>	Error occurred	<code>req.on("error", function)</code>

async/await

- Suppose you want to read two files and display their contents. Display File1 content first & then file 2 content. Ordering of output is important.

```
const fs = require("fs");
fs.readFile("file1.txt", "utf8", (err, data1) => {
  if (err) throw err;
  console.log("Student Details:");
  console.log(data1);
  fs.readFile("file2.txt", "utf8", (err, data2) => {
    if (err) throw err;
    console.log("Marks Details:");
    console.log(data2);
  });
});
```

async/await

Problem with callback approach

1. Nested structure (callback inside callback)
2. Hard to read as program grows
3. Difficult to debug
4. Becomes callback hell if more steps are added

- `async` and `await` are keywords in Node.js (JavaScript) used to handle asynchronous operations in a simple way.
- `async` → makes a function return a Promise
- `await` → makes the function wait for a result before moving to next line

WHY?

- Node.js operations (file, DB, API) take time
- Callbacks make code complex (nested)
- `async/await` makes code simple and readable

async/await

```
const fs = require("fs").promises;
async function displayDetails() {
  try {
    const data1 = await fs.readFile("file1.txt", "utf8");
    console.log("Student Details:");
    console.log(data1);
    const data2 = await fs.readFile("file2.txt", "utf8");
    console.log("Marks Details:");
    console.log(data2);
  } catch (err) {
    console.log("Error:", err);
  }
}
displayDetails();
```

Database Connectivity

- Establishing a connection between a Node.js application and a MongoDB database.
 1. Create Project Folder
 2. Go to project folder and run `npm init -y`
(This creates `package.json` & used to initialize project)
 3. Install mongodb driver using `npm install mongodb`
 4. `mongod` (Install mongodb community edition and mongodb shell)
 5. `mongosh` (open mongodb shell)
 6. In cmd prompt `net start MongoDB` It starts the MongoDB Windows service. (`mongod` → runs in background)
 7. Then type `mongosh` it connect with mongodb server
(`mongodb://127.0.0.1:27017`)

Database Connectivity

Basic Steps for Connectivity using node.js

- 1.Import MongoDB client
- 2.Create a connection URL
- 3.Create a client instance
- 4.Connect to database
- 5.Perform operations
- 6.Close connection

Database Connectivity

1. `const { MongoClient } = require('mongodb');`

- `require('mongodb')` loads the MongoDB Node.js Driver module.
- Imports **MongoClient** client only from `mongodb` package
- **MongoClient** is the core interface used to connect Node.js with MongoDB.

2. `const url = "mongodb://localhost:27017";`

- This is `mongodb` connection string url
- `mongodb://` → protocol used by `mongodb`
- `localhost:27017` → default `mongodb` port no.

3. `const client = new MongoClient(url);`

- Instantiates the `MongoClient` class.

Database Connectivity

4. Define an async function (non-blocking I/O).

```
async function connectDB()
{
  await client.connect();
  console.log("Connected to MongoDB"); //
  connectDB(); //Calls the async function.
  const db= client.db("Database");
  return db;
}
```

```
await client.close();
```

```
try {
  await client.connect();
}
catch (err) {
  console.error(err);
}
```

Database Connectivity

```
const http = require('http');
const { MongoClient } = require('mongodb');
const url = "mongodb://127.0.0.1:27017";
const client = new MongoClient(url);
const server= http.createServer(async (req, res) => {
  try
  {
    await client.connect();
    const db = client.db("taskAppDB");
    const collection = db.collection("users");
    await collection.insertOne({
      name: "Alice", email: "alice@example.com", role: "Admin"
    });
  }
});
```

Database Connectivity

```
const data = await collection.findOne({ name: "Alice" });
res.writeHead(200, { 'Content-Type': 'text/html' });
  res.write(`<h2>User Details</h2>
    <p>Name: ${data.name}</p>
    <p>Email: ${data.email}</p>
    <p>Role: ${data.role}</p>`);
  res.end();
}
catch (err) {
  res.write("Error: " + err);
  res.end();
}
})
Server.listen(3000, () => { console.log("Server running"); });
```

Express.js

Express.js is a lightweight web framework for Node.js used to build web applications and APIs easily.

Used to handle

- i. HTTP requests
- ii. Routing (URL handling)
- iii. Middleware logic (process request before → route handler)
- iv. Server setup in a simple way
- v. Used to build REST APIs.

Installation

```
npm install express
```

Express.js

```
const express = require('express');  
const app = express(); //creates an express application object
```

```
app.get('/', (req, res) => {  
  res.send("Hello World from Express!");  
});
```

```
app.listen(3000, () => {  
  console.log("Server running on port 3000");  
});
```